IESB - Programacao Paralela (CCO085)

Desafio Final 009

Analise de Dados com OpenMP

Relatorio de implementacao, resultados estatisticos e desempenho

Identificacao	Informacao confirmada
Disciplina	Programacao Paralela (CCO085)
Instituicao / periodo	IESB / 2026-2
Professor	Rodrigo Goncalves Pinto
Projeto	Exercicio Final: Analise de Dados com OpenMP
Integrantes	Nao informados nos arquivos versionados
Repositorio	github.com/FilipeCampos25/Desafio_final_openmp

Brasilia - novembro de 2026

1. Objetivo

O objetivo e calcular a media ponderada, o desvio padrao populacional e o coeficiente de variacao (CV) da estatura e do peso de 1000 pessoas, agrupando os valores em classes. O exercicio tambem compara uma implementacao sequencial com uma implementacao paralela em C++17/OpenMP, verificando corretude e desempenho.

2. Dados e intervalos de classe

A entrada usada possui exatamente 1000 pares estatura-peso. Ela e sintetica, deterministica, gerada com semente 20261110 e nao foi fornecida pelo professor. Portanto, os numeros deste relatorio nao devem ser interpretados como estatisticas de uma populacao real ou de uma base oficial.

As classes sao semiabertas [limite inferior, limite superior). A estatura usa largura de 8 cm e apresenta classes observadas de [144, 152) ate [192, 200). O peso usa largura de 4 kg e apresenta classes de [44, 48) ate [116, 120). O ponto medio xi representa os fi registros de cada classe; a soma das frequencias e 1000 em ambas as variaveis.

3. Formulas estatisticas

Para classes com ponto medio xi, frequencia fi e N = 1000, foram aplicadas as formulas de dados agrupados:

```
media = sum(fi * xi) / N
sigma = sqrt(sum(fi * (xi - media)^2) / N)
CV(%) = 100 * sigma / media
```

O denominador da variancia e N, logo o desvio e populacional, nao amostral. O CV e adimensional e permite comparar a dispersao relativa de variaveis com unidades e medias diferentes.

4. Implementacao sequencial

A versao sequencial le e valida os 1000 registros, cria histogramas ordenados, calcula a soma ponderada, determina a media e percorre novamente as classes para somar os desvios quadraticos ponderados. Todas as etapas usam um unico fluxo de execucao.

5. Implementacao OpenMP

Na versao paralela, um parallel for calcula o indice de classe de cada registro. Reducoes min e max determinam a faixa dos indices. Em seguida, cada thread atualiza um histograma privado, eliminando corridas nas frequencias. A consolidacao percorre as classes em paralelo; cada iteracao escreve em uma posicao final exclusiva.

As somas da media e da variancia usam reduction(+:acumulador). O OpenMP cria acumuladores privados por thread e combina os valores ao final da regiao, evitando atualizacoes concorrentes no mesmo endereco. Esses lacos podem ser paralelizados porque cada registro ou classe e processado independentemente; somente os totais precisam ser combinados. O agendamento padrao divide as iteracoes do for entre as threads da equipe e aplica uma barreira implicita ao final.

6. Corretude e resultados estatisticos

Antes do benchmark, os testes automatizados validaram formato e quantidade da entrada, fronteiras das classes, frequencias, casos invalidos e uma referencia Python independente. O OpenMP foi executado repetidamente com 1, 2 e 4 threads. Todas as configuracoes coincidiram com a versao sequencial dentro da tolerancia definida.

Variavel	Media ponderada	Desvio populacional	CV (%)
Estatura	172,608000 cm	15,964408 cm	9,248939
Peso	82,420000 kg	21,799073 kg	26,448767

O peso apresenta dispersao relativa maior: seu CV e cerca de 26,45%, contra 9,25% para a estatura na base sintetica.

7. Metodologia do benchmark

O script scripts/benchmark.py usa time.perf_counter_ns(), relogio monotonico de alta resolucao. Foram realizados tres aquecimentos nao registrados e 30 ensaios por configuracao: sequencial e OpenMP com 1, 2 e 4 threads. A ordem das configuracoes foi rotacionada entre rodadas para reduzir vies sistematico de aquecimento.

O tempo inclui a criacao do processo, inicializacao, leitura do mesmo arquivo pessoas.txt, construcao dos histogramas, calculos e escrita da saida capturada. OMP_DYNAMIC foi desativado. Cada execucao tambem teve suas estatisticas conferidas antes de o tempo ser aceito.

8. Comparacao de tempos

Configuracao	n	Media (ms)	Mediana (ms)	DP (ms)	Speedup
Sequencial	30	16.856280	16.521450	1.672749	1.000000x
OpenMP 1 thread	30	17.140990	16.895100	0.651562	0.983390x
OpenMP 2 threads	30	18.322650	17.829400	2.090454	0.919970x
OpenMP 4 threads	30	18.348070	17.937350	1.651019	0.918695x

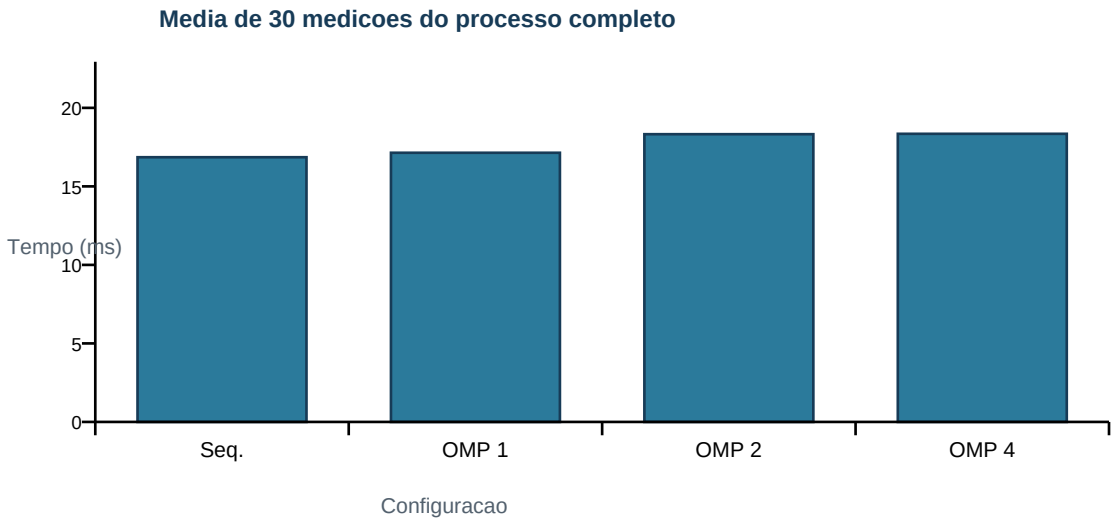

Figura 1 - Medias reais registradas em benchmark_medicoes.csv. Barras menores indicam menor tempo.

9. Analise de desempenho e limitacoes

Nenhuma configuracao OpenMP obteve speedup maior que 1. Com uma thread, o tempo medio foi 1,66% superior ao sequencial. Com duas e quatro threads, os tempos foram aproximadamente 8,70% e 8,85% superiores. O resultado e esperado para apenas 1000 registros e poucas classes: o trabalho aritmetico de cada laco e pequeno, enquanto cada regiao paralela exige formar a equipe, distribuir iteracoes, sincronizar barreiras e combinar reducoes.

A medicao do processo completo e representativa do uso do programa, mas dilui o trecho computacional no custo de criacao do processo, leitura e saida. O sistema operacional pode introduzir variacao por escalonamento, frequencia do processador, cache e tarefas em segundo plano. A amostra sintetica e pequena; cargas maiores, mais classes ou mais calculos por registro podem alterar o ponto de equilibrio. Tambem nao se deve generalizar os tempos deste host para Docker ou outros compiladores.

Ambiente medido

Item	Valor registrado
Sistema	Windows-10-10.0.26200-SP0
Processador	AMD Ryzen 5 4500 6-Core Processor
CPUs logicas	12
Compilador	g++.exe (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r3) 16.2.0
Flags	-O2 -std=c++17 -Wall -Wextra -Wpedantic; OpenMP: -fopenmp
Docker	indisponivel no PATH; benchmark fora de Docker
Entrada SHA-256	16472c57436e244488a67da42d87bf74b4b4f412fa96c3af06e08c4514dc8c6a

10. Conclusao

As implementacoes sequencial e OpenMP calcularam corretamente as estatisticas agrupadas da base sintetica. A paralelizacao e segura porque usa iteracoes independentes, histogramas privados e reducoes. Entretanto, neste tamanho de entrada, a sobrecarga foi maior que o trabalho economizado, e a versao sequencial apresentou o menor tempo medio. O resultado demonstra que corretude e paralelismo nao garantem aceleracao: granularidade e custo de coordenacao precisam ser considerados.

11. Reprodutibilidade

Os comandos make test e make benchmark recompilam, validam e medem o projeto. Os tempos individuais, o resumo, as estatisticas, hashes e o ambiente estao em resultados/. Este PDF pode ser recriado com python3 scripts/gerar_relatorio.py. O README descreve todo o fluxo.

Arquivos consultados

CHECKLIST.md; README.md; Dockerfile; Makefile; dados/README.md; include/dados.hpp; src/dados.cpp; cv_sequencial.cpp; cv_openmp.cpp; tests/; resultados/benchmark_resumo.csv; resultados/benchmark_ambiente.txt; resultados/estatisticas.csv.